OPIPE-01: OpenPipeline as a Multi-Scope Platform

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 1 of 7 | **Created:** March 2026 | **Last Updated:** 05/06/2026

Beyond Logs: Processing Spans, Metrics, and Events at Ingestion

OpenPipeline is often introduced as a log processing framework — and logs are indeed the most common entry point. But OpenPipeline operates on **six distinct scopes**, each with its own processing pipeline, routing rules, and extraction capabilities. This notebook builds intuition for how the platform works across all scopes and why pipeline design matters.

> **Companion Series**
> - **OPLOGS** — Deep-dive into log processing: **OPLOGS-01: OpenPipeline Fundamentals**
> - **OPMIG** — Migrating from classic logs? Start with **OPMIG-01: Why Migrate**
> - **SPANS** — Querying and analyzing distributed traces: **SPANS-01: Fundamentals**

Table of Contents

1. The Six OpenPipeline Scopes
2. Shared Architecture Across Scopes
3. The Default Pipeline Anti-Pattern
4. Pipeline Design Principles

5. Processing Groups: Conditional Logic Within a Pipeline
6. Security Context Across Scopes
7. Ingestion-Time vs. Query-Time Processing
8. Summary
9. Next Steps
10. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:logs:read`, `storage:spans:read`, `storage:metrics:read`, `storage:events:read`, `storage:bizevents:read`
Recommended	Familiarity with **OPLOGS-01** (log pipeline basics)

Sprint 1.337 (April 2026): OneAgent Primary Fields/Tags as Routing Keys

Sprint 1.337 SaaS landed a major OpenPipeline-relevant change: OneAgent now enriches **all** telemetry (spans, metrics, logs, business events, Smartscape entities) at the source with standardized **primary fields** and customer-defined **primary tags** as top-level attributes on Latest Dynatrace tenants.

Primary fields (Semantic Dictionary-defined):

Field	Purpose
`dt.security_context`	ABAC scope for IAM policies; also drives bucket-routing decisions
`dt.cost.costcenter`	Cost allocation tag; routes spend to org units
`dt.cost.product`	Product-line attribution for cost rollups

Primary tags (customer-defined): set during OneAgent install via `oneagentctl --set-host-tag=<key>=<value>` — e.g., `team`, `env`, `app`, `data_classification`. Land as `primary_tags.<key>` on every signal.

Pipeline implication for cross-scope design:

1. **Routing rules** can dispatch on primary fields directly without parse processors:

`yaml processors: - type: route rules: - condition: "dt.cost.costcenter == 'cc-1234'" destination: "finance_logs" - condition: "dt.security_context contains 'pci'" destination: "pci_audit_logs_365d"`

2. **Cross-scope consistency:** because primary fields/tags appear on logs, spans, metrics, AND business events, queries that join across scopes (OPIPE-06 cross-scope design patterns) can correlate without scope-specific lookup tables.
3. **Non-OneAgent sources** (raw syslog, third-party log shippers, OTLP-via-collector) still need OpenPipeline `enrichment` processors to surface the same fields. Document this split in your standard.

Sprint-337 also added recommended-field suggestions to extraction processors — the UI flags permission-relevant fields and Smartscape identifiers in the field-promotion dialog, preventing accidental promotion of sensitive content. Existing extraction processors keep working unchanged.

---

1. The Six OpenPipeline Scopes

OpenPipeline processes data through **six independent scopes**. Each scope has its own set of pipelines, routing rules, processors, and storage targets. They share the same architecture but operate on different data types.

Scope	Data Object	Primary Use Case	Key Fields
Logs	`logs`	Application and infrastructure log records	`content`, `loglevel`, `log.source`
Spans	`spans`	Distributed trace spans from services	`span.kind`, `trace.id`, `duration`

Scope	Data Object	Primary Use Case	Key Fields
Metrics	`timeseries`	Time-series measurements from agents and extensions	`metric.key`, `dt.entity.*`
Events	`events`	Lifecycle and configuration change events	`event.kind`, `event.type`
Business Events	`bizevents`	Business transactions and user actions	`event.type`, `event.provider`
Security Events	`events` (security bucket)	Threat detection and audit records	`event.kind`, `security.category`

What This Means in Practice

Each scope is configured **independently** in the Dynatrace UI under **OpenPipeline** settings. A processing rule you create for logs has no effect on spans. A routing rule for business events does not apply to metrics. This independence is a feature — it means you can evolve each scope's pipeline at its own pace without risk to other data types.

Discovering Your Active Scopes

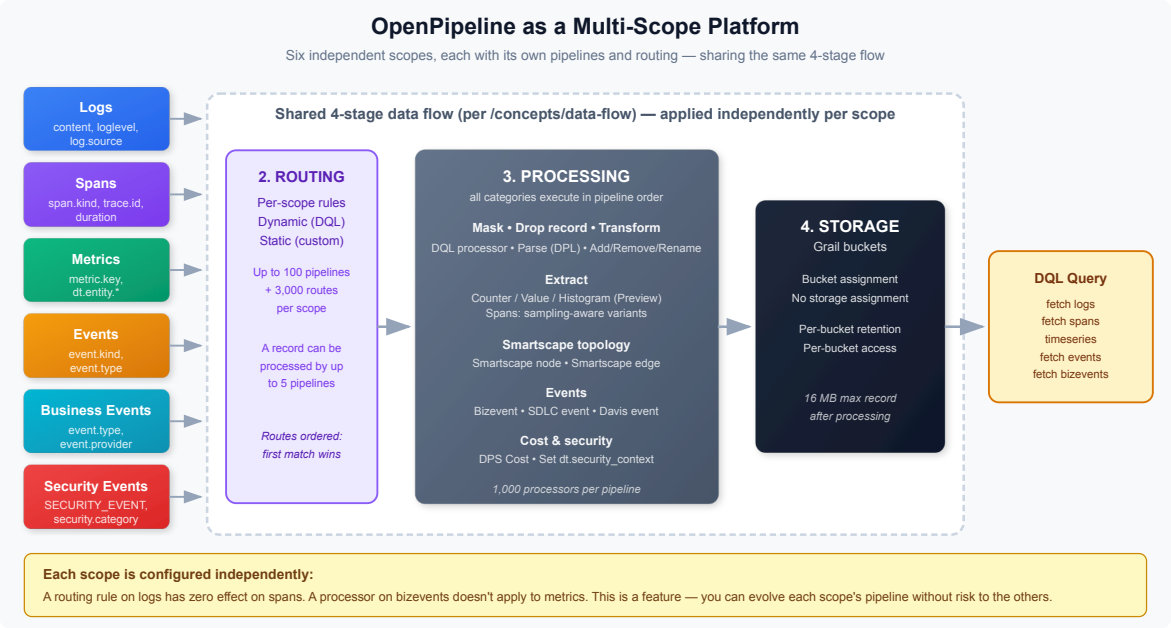
The following queries show what data is flowing through each scope in your environment.

```
// Log pipelines: volume by pipeline and bucket
fetch logs, from:-1h
| summarize log_count = count(), by:{dt.openpipeline.pipelines,
dt.system.bucket}
| sort log_count desc
```

```
// Span pipelines: volume by pipeline and bucket
fetch spans, from:-1h
| summarize span_count = count(), by:{dt.openpipeline.pipelines,
dt.system.bucket}
| sort span_count desc
```

```
// Business event pipelines: volume by provider and type
fetch bizevents, from:-1h
| summarize event_count = count(), by:{event.provider, event.type}
| sort event_count desc
```

2. Shared Architecture Across Scopes



Every scope follows the same **four-stage data flow** (Ingest → Routing → Processing → Storage), with optional pre-processing on custom sources. The stages are identical in concept across scopes — only the data types and available processors differ.

Stage	Purpose	Logs Example	Spans Example
1. Ingest	Receive records from a built-in / ready-made / custom source	OneAgent log ingest	OTLP span ingest
2. Routing	Direct data to the right pipeline (dynamic DQL match or static for custom sources)	Match by `log.source` or `k8s.namespace.name`	Match by `span.kind` or `service.name`

Stage	Purpose	Logs Example	Spans Example
3. Processing	All in-pipeline work — Mask, Drop, Transform/Parse, Extract (counter / value / histogram / Smartscape / event), Cost, Security, Bucket assignment	Mask credit card numbers; drop debug; parse JSON; extract metrics	Mask PII; drop health-check spans; sampling-aware metrics; extract RED metrics
4. Storage	Persist to a Grail bucket (or skip with No storage assignment)	Send to `app_logs` bucket	Send to `trace_data` bucket

> **Doc alignment (May 2026):** Per `/concepts/data-flow`, masking, filtering, transformation, and extraction are **processor categories within the Processing stage** — not separate pipeline stages. Earlier versions of this notebook described a six-stage pipeline; the corrected four-stage flow is shown above. The processor execution order *within* Processing is documented in OPMIG-02 § Processing Order.

The Key Insight

If you understand how to build a log pipeline (route → mask → drop → transform → extract → assign bucket within Processing), you already understand how to build a span pipeline or an event pipeline. The concepts transfer directly — only the field names and processor options change.

Processors Available Per Scope

Not every processor is available in every scope. The following table shows key differences (refresh against `/concepts/processing` for the current list):

Processor	Logs	Spans	Metrics	Events	Bizevents
DPL parsing	Yes	Yes	—	Yes	Yes
Field enrichment (Add / Remove / Rename / DQL)	Yes	Yes	Yes	Yes	Yes

Processor	Logs	Spans	Metrics	Events	Bizevents
Metric extraction (Counter / Value / Histogram Preview)	Yes	Yes (sampling-aware variants)	—	Yes	Yes
Smartscape node / edge	Yes	Yes	Yes	Yes	Yes
Event extraction (Bizevent / SDLC / Davis)	Yes	Yes	—	—	—
Masking	Yes	Yes	—	Yes	Yes
Drop record	Yes	Yes	Yes	Yes	Yes
Bucket assignment / No storage assignment	Yes	Yes	Yes	Yes	Yes
Set dt.security_context	Yes	Yes	Yes	Yes	Yes
DPS Cost Allocation (Cost Center / Product)	Yes	Yes	Yes	Yes	Yes

OneAgent Attribute Enrichment (OneAgent 1.331+)

> **Requires:** OneAgent version **1.331** or later

OneAgent can enrich **all telemetry signals** (metrics, spans, logs, events, entities) with custom metadata at the source — before data reaches the Dynatrace platform. This is more efficient than server-side tagging (auto-tags) because enrichment happens on the host and propagates to all Smartscape nodes.

Primary Fields (standardized from Semantic Dictionary):
- `dt.security_context` — data governance and access control
- `dt.cost.costcenter` — cost allocation
- `dt.cost.product` — product attribution

Primary Tags (custom key-value pairs):
- `primary_tags.environment` — environment identification (production, staging, etc.)
- `primary_tags.team` — team ownership
- `primary_tags.business_unit` — organizational unit

Configuration:

```
# Set during OneAgent installation
Dynatrace-OneAgent-Linux.sh --set-host-
tag="primary_tags.environment=production" --set-host-
tag="dt.security_context=confidential"

# Set on existing agents via oneagentctl
oneagentctl --set-host-tag="primary_tags.environment=production"
oneagentctl --set-host-tag="dt.cost.costcenter=12345"

# Per-process via environment variable (overrides host-level)
DT_TAGS="primary_tags.team=platform primary_tags.environment=production"
```

Benefits over auto-tagging:

Aspect	Auto-Tags (Server-Side)	Attribute Enrichment (Agent-Side)
When applied	After data arrives at platform	At the source, before transmission
Scope	Entity tags only	All signals: metrics, spans, logs, events, entities
Grail integration	Limited	Full — feeds OpenPipeline routing, bucket assignment, permissions
Cost allocation	Not supported	`dt.cost.costcenter`, `dt.cost.product` fields
Security context	Not supported	`dt.security_context` for data governance

> **See:** Primary Grail fields and tags enrichment through OneAgent

3. The Default Pipeline Anti-Pattern

Every OpenPipeline scope ships with a **Default Pipeline**. It is a catch-all: any data that does not match a custom pipeline's routing rules flows through the default pipeline and lands in the default bucket (e.g., `default_logs`, `default_spans`).

This is convenient for getting started. It becomes a serious problem at scale.

Why "Everything in Default" Fails

Problem	Impact
Query performance	Every DQL query against the default bucket scans ALL data — application logs mixed with infrastructure noise, health checks mixed with business transactions. The 500 GB scan limit becomes a wall.

Problem	Impact
Cost	No differentiated retention. You pay to store debug-level container stdout at the same retention tier as critical security audit logs.
Security	No `dt.security_context` separation. Without dedicated pipelines, you cannot apply IAM policies to restrict who sees which data. Finance team logs are visible to infrastructure engineers and vice versa.
Blast radius	A misconfigured processing rule (bad parsing regex, aggressive drop filter) in the default pipeline affects ALL data flowing through it. One mistake breaks everything.
No metric extraction	You cannot extract targeted metrics from specific log patterns when everything is in one undifferentiated stream. Metric extraction rules become overly complex or extract noise.
Debugging	When something goes wrong, you cannot tell which data source caused the issue. Pipeline-level monitoring (`dt.openpipeline.pipelines`) shows a single pipeline name for everything.

The Telltale Signs

Run the following queries to check if your environment has this problem.

```
// Check: What percentage of logs land in the default bucket?
fetch logs, from:-1h
| summarize total = count(),
    default_count = countIf(dt.system.bucket == "default_logs")
| fieldsAdd default_pct = round(toDouble(default_count) / toDouble(total) *
100, decimals: 1)
| fieldsAdd assessment = if(default_pct &gt; 80, then: "CRITICAL: Most data
in default bucket",
    else: if(default_pct &gt; 50, then: "WARNING: Majority in default
bucket",
    else: "OK: Data is distributed across buckets"))
```

```
// Check: How many distinct pipelines are processing your logs?
fetch logs, from:-1h
| summarize pipeline_count = countDistinct(dt.openpipeline.pipelines),
    bucket_count = countDistinct(dt.system.bucket),
    source_count = countDistinct(dt.openpipeline.source)
| fieldsAdd assessment = if(pipeline_count &lt;= 1, then: "CRITICAL: Only
default pipeline in use",
    else: if(pipeline_count &lt; 3, then: "WARNING: Very few pipelines
configured",
    else: "OK: Multiple pipelines configured"))
```

```
// Visualize: Volume distribution across buckets (are they balanced?)
fetch logs, from:-1h
| summarize log_count = count(), by:{dt.system.bucket}
| sort log_count desc
| fieldsAdd pct_of_total = round(toDouble(log_count) /
toDouble(arraySum(log_count)) * 100, decimals: 1)
```

```
// Same check for spans: are they all in default_spans?
fetch spans, from:-1h
| summarize total = count(),
    default_count = countIf(dt.system.bucket == "default_spans")
| fieldsAdd default_pct = round(toDouble(default_count) / toDouble(total) *
100, decimals: 1)
| fieldsAdd assessment = if(default_pct &gt; 80, then: "CRITICAL: Most spans
in default bucket",
    else: if(default_pct &gt; 50, then: "WARNING: Majority in default
bucket",
    else: "OK: Spans are distributed across buckets"))
```

4. Pipeline Design Principles

The goal is to move from "everything in default" to a **purpose-driven pipeline architecture**. Here are the principles:

Principle 1: One Pipeline Per Source Type

Create separate pipelines based on the nature of the data, not the team that owns it.

Pipeline	Routing Condition	Target Bucket	Retention
`kubernetes-infra`	`k8s.namespace.name` exists AND `log.source == "Container Output"`	`k8s_infra_logs`	14 days
`application-logs`	`log.source == "Log file"` AND `k8s.namespace.name` matches app namespaces	`app_logs`	35 days
`security-audit`	`loglevel == "AUDIT"` OR `log.source` matches security sources	`security_audit`	365 days
`api-ingestion`	`dt.openpipeline.source == "/api/v2/logs/ingest"`	`external_logs`	35 days
Default Pipeline	Everything else (catch-all)	`default_logs`	14 days

Principle 2: Filter Early, Extract Targeted

Each pipeline should drop noise **before** storage and extract **only** the metrics relevant to that data type:

- **kubernetes-infra pipeline**: Drop health check logs, extract pod restart counts
- **application-logs pipeline**: Drop DEBUG in production, extract error rates by service
- **security-audit pipeline**: Never drop, never sample — extract compliance event counts

Principle 3: Differentiate Retention

Different data has different value over time:

Data Type	Suggested Retention	Rationale
Debug/container stdout	7-14 days	High volume, short-term troubleshooting
Application logs	35 days	Standard operational window

Data Type	Suggested Retention	Rationale
Security/audit logs	365+ days	Compliance requirements (SOC 2, HIPAA)
Business events	90-365 days	Trend analysis, quarterly reporting

Principle 4: Route Ordering Matters

Pipeline routing rules are evaluated **in order — first match wins**. Place specific rules before general ones:

1. Security audit logs (most specific, highest value)
2. Application logs by namespace
3. Kubernetes infrastructure
4. API-ingested external logs
5. Default pipeline (catch-all for anything unmatched)

Applying This to Spans

The same principles apply to the spans scope:

Pipeline	Routing Condition	Target Bucket
`frontend-traces`	`span.kind == "server"` AND `service.name` matches frontend services	`frontend_spans`
`backend-traces`	`span.kind == "server"` AND `service.name` matches backend services	`backend_spans`
`database-spans`	`span.kind == "client"` AND `db.system` exists	`db_spans`
Default Pipeline	Everything else	`default_spans`

```
// Inventory: What log sources exist and how much volume do they produce?
// Use this to plan your pipeline routing rules
fetch logs, from:-24h
| summarize log_count = count(),
    unique_hosts = countDistinct(dt.entity.host),
    by:{dt.openpipeline.source, log.source}
| sort log_count desc
| limit 20
```

```
// Inventory: Kubernetes namespace volume (for pipeline routing decisions)
fetch logs, from:-24h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name}
| sort log_count desc
| limit 20
```

```
// Inventory: Span volume by service (for span pipeline routing decisions)
fetch spans, from:-24h
| summarize span_count = count(), by:{service.name, span.kind}
| sort span_count desc
| limit 20
```

5. Processing Groups: Conditional Logic Within a Pipeline

Sections 3 and 4 explained why you need separate pipelines and how to design them. But what happens when a single pipeline receives data from multiple sources that need **different processing logic** — without justifying an entirely new pipeline?

This is what **processing groups** solve.

What Is a Processing Group?

A processing group is a named collection of processors within a pipeline stage that share a **common matching condition**. Only records that satisfy the group's matcher are processed by the group's processors. Records that don't match skip the group entirely.

```
Pipeline: application-logs
├── Processing Stage
│   ├── Group: "java-apps"     (matcher: k8s.namespace.name == "java-prod")
│   │   ├── Processor: Parse Java stack traces
│   │   ├── Processor: Extract exception class
│   │   └── Processor: Add team = "backend"
│   │
│   ├── Group: "nginx-access"  (matcher: log.source == "nginx")
│   │   ├── Processor: Parse access log format
│   │   ├── Processor: Extract response code + latency
│   │   └── Processor: Add team = "platform"
│   │
│   └── Global Processor: Add environment = "production"  (no matcher — runs
on ALL records)
```

How Processing Groups Work

Behavior	Detail
Matching	Each group has a DQL matching condition. Only records satisfying the condition enter the group.
Inheritance	All processors within a group inherit the group's matcher. You do not repeat the condition on each processor.
Multi-match	If a record matches multiple groups, **all matching groups execute**. Groups are independent — they do not short-circuit.
Execution order	Processors **within** a group execute in the order they are listed.
Global processors	Processors placed outside any group run on **all records** unconditionally — before or after groups, depending on their position.
Available in all scopes	Processing groups work in Logs, Spans, Metrics, Events, and Business Events scopes.

When to Use Processing Groups vs. Separate Pipelines

Scenario	Use Processing Groups	Use Separate Pipelines
Same bucket destination, different parsing	**Yes** — group per log format	No

Scenario	Use Processing Groups	Use Separate Pipelines
Different retention requirements	No	**Yes** — different buckets
Different security context	No	**Yes** — different `dt.security_context` assignment
Same source, multiple enrichment paths	**Yes** — group per enrichment path	No
Completely different data lifecycle	No	**Yes** — separate routing, storage, retention
Reducing pipeline proliferation	**Yes** — consolidate related logic	N/A
Team-level blast radius isolation	No	**Yes** — a bad processor in one pipeline can't affect another

The Decision Rule

> **Use processing groups** when the data shares the same pipeline-level concerns (routing, bucket, retention, security context) but needs **different processing logic** based on content.
>
> **Use separate pipelines** when the data needs different **lifecycle treatment** — different buckets, different retention, different security context, or different teams owning the configuration.

Example: Multi-Format Log Pipeline

A single `application-logs` pipeline receives logs from Java services, Node.js services, and Python services. Each has a different log format, but they all go to the same `app_logs` bucket with 35-day retention.

Without processing groups, you would need 3 separate pipelines — tripling the configuration surface and the number of routing rules.

With processing groups:

Group	Matching Condition	Processors
`java-logs`	`matchesValue(k8s.container.name, "java-*")`	Parse log4j format, extract exception class, enrich with `app.framework = "java"`
`nodejs-logs`	`matchesValue(k8s.container.name, "node-*")`	Parse JSON structured logs, extract request ID, enrich with `app.framework = "nodejs"`
`python-logs`	`matchesValue(k8s.container.name, "python-*")`	Parse Python logging format, extract traceback, enrich with `app.framework = "python"`
(global)	All records	Add `dt.security_context = "app-team"`, drop records where `loglevel == "DEBUG"`

The global processors handle what's common to all records. The groups handle what's specific to each format. One pipeline, one bucket, three parsing paths.

Example: Span Enrichment by Service Category

In the Spans scope, a single pipeline processes all server spans but needs different enrichment based on service type:

Group	Matching Condition	Processors
`api-services`	`matchesValue(http.route, "/api/*")`	Add `span.category = "api"`, extract API version from path
`database-calls`	`isNotNull(db.system)`	Add `span.category = "database"`, normalize `db.statement` to remove parameters
`messaging`	`in(span.kind, {"producer", "consumer"})`	Add `span.category = "messaging"`, extract queue name

Common Mistakes

Mistake	Problem	Fix
Duplicating matchers on every processor	Verbose, error-prone, hard to maintain	Use a processing group — define the matcher once
Overlapping group matchers without intent	A record processed by multiple groups may get conflicting field values	Make matchers mutually exclusive, or design for intentional multi-match
Putting bucket routing in a group	Bucket assignment applies to the whole pipeline, not per-group	Configure bucket routing at the pipeline level, not inside groups
Too many groups in one pipeline	Becomes as complex as having separate pipelines	If you have >5-6 groups, consider splitting into separate pipelines

6. Security Context Across Scopes

The `dt.security_context` field controls **who can see which data** through IAM policies. It is the mechanism that makes pipeline separation actionable from a governance perspective.

How Security Context Works

When data flows through an OpenPipeline, you can enrich it with a `dt.security_context` value. IAM policies then use this value to restrict query access:

```
Data Ingestion → OpenPipeline adds dt.security_context → Grail stores it →
IAM policy enforces access
```

For a complete deep-dive on security context patterns, values, and IAM policy syntax, see **ORGNZ-06: Security Context**.

Security Context by Scope

Scope	How to Set `dt.security_context`	Notes
Logs	OpenPipeline field enrichment processor	Most common — set based on `k8s.namespace.name`, `log.source`, or parsed fields
Spans	OpenPipeline field enrichment processor	Set based on `service.name` or span attributes
Metrics	OpenPipeline field enrichment processor **or** extension configuration	See "Extension Metrics" below
Events	OpenPipeline field enrichment processor	Set based on `event.type` or entity association
Business Events	OpenPipeline field enrichment processor	Set based on `event.provider` or business unit
Entities	Entity properties via Settings API	Set on the entity itself, inherited by associated data

Extension Metrics and Security Context

Metrics ingested via **Extensions 2.0** do not automatically inherit `dt.security_context`. This is a common gap — teams deploy custom extensions to collect database metrics, cloud service metrics, or infrastructure data, and assume the security context is handled automatically.

It is not. You must explicitly configure it.

There are two approaches:

Approach 1: OpenPipeline Metric Enrichment (Recommended)

Configure the **Metrics** scope in OpenPipeline to add `dt.security_context` based on dimensions that identify the source:

Routing Condition	Security Context Value	Use Case
`metric.key` starts with `ext:com.dynatrace.extension.database`	`"database-team"`	Database monitoring extensions
`metric.key` starts with `ext:com.dynatrace.extension.cloud`	`"cloud-team"`	Cloud integration extensions
`dt.entity.host` matches specific host group	`"team-a"`	Team-scoped infrastructure metrics

Approach 2: Entity-Level Security Context

Set `dt.security_context` on the **entities** that the extension monitors (hosts, process groups). Metrics associated with those entities inherit the context through entity relationships. This is less granular but simpler to manage.

> **Structured context design:** For organisations with transversal teams (database, networking, OS) that need cross-application access, a flat context value (e.g., `"checkout"`) does not scale. Encoding `comp:<component>/bu:<business-unit>/app:<application>` into the context string, then using `MATCH('comp:db*')` in the IAM policy, enables precise transversal access without a per-application context per team. The dimension needed for transversal slicing should come first in the string for compatibility with Classic entity `startsWith` matching. See **IAM-04: Policy Authoring** for the complete design pattern and **IAM-05: Boundary Design** for boundary examples.
>
> **Critical:** When `dt.security_context` holds an array value, IAM policies must use the `MATCH` operator — not `=`, `STARTSWITH`, or `IN`. Array comparison with equality operators always returns false. See **ORGNZ-06** for the full rules.

```
// Check: Do your logs have security context assigned?
fetch logs, from:-1h
| summarize total = count(),
    with_context = countIf(isNotNull(dt.security_context)),
    without_context = countIf(isNull(dt.security_context))
| fieldsAdd coverage_pct = round(toDouble(with_context) / toDouble(total) *
100, decimals: 1)
```

```
// Check: Do your spans have security context assigned?
fetch spans, from:-1h
| summarize total = count(),
    with_context = countIf(isNotNull(dt.security_context)),
    without_context = countIf(isNull(dt.security_context))
| fieldsAdd coverage_pct = round(toDouble(with_context) / toDouble(total) *
100, decimals: 1)
```

7. Ingestion-Time vs. Query-Time Processing

A critical design decision: should you process data in OpenPipeline (at ingestion) or in DQL (at query time)? The answer depends on what you are trying to achieve.

When to Process at Ingestion (OpenPipeline)

Use Case	Why Ingestion Time
Drop noise	Data you never want to see should never be stored. Dropping debug logs, health check spans, or synthetic test events saves cost permanently.
Mask sensitive data	PII must be redacted before storage for compliance. You cannot "un-store" data.
Add security context	`dt.security_context` must be set at ingestion — it cannot be added retroactively.
Extract metrics	Converting log patterns or span attributes into metrics happens at ingestion. You cannot create metrics from historical data.
Route to buckets	Bucket assignment is permanent at ingestion time. You cannot move data between buckets after storage.

When to Process at Query Time (DQL)

Use Case	Why Query Time
Exploratory analysis	When you do not yet know what you are looking for, keep the raw data and query flexibly.

Use Case	Why Query Time
Changing requirements	If the transformation logic changes frequently, DQL queries are easier to update than pipeline configurations.
Ad-hoc parsing	One-time investigations benefit from DQL `parse` rather than permanent pipeline processors.
Complex joins	Correlating across multiple data sources is a query-time operation.
Aggregation and visualization	Dashboard queries, trend analysis, and statistical aggregations belong in DQL.

The Decision Framework

Ask these questions in order:

1. **Is the data sensitive?** → Mask at ingestion. No exceptions.
2. **Is the data noise?** → Drop at ingestion. Do not pay to store it.
3. **Does it need access control?** → Add security context at ingestion.
4. **Do you need a metric from it?** → Extract at ingestion.
5. **Everything else?** → Store raw, process at query time.

Summary

In this notebook you learned:

- **Six scopes** — OpenPipeline processes logs, spans, metrics, events, business events, and security events independently
- **Shared architecture** — All scopes follow the same four-stage flow (ingest → routing → processing → storage); masking, filtering, transformation, and extraction are processor categories within the Processing stage per `/concepts/data-flow`
- **The default pipeline anti-pattern** — Sending everything through the default pipeline causes query performance, cost, security, and blast radius problems
- **Pipeline design principles** — One pipeline per source type, filter early, differentiate retention, order routing rules from specific to general

- **Processing groups** — Conditional logic within a pipeline: group processors by matching condition to handle multiple data formats without creating separate pipelines. Use groups for different processing; use pipelines for different lifecycle.
- **Security context across scopes** — `dt.security_context` must be set at ingestion; extension metrics require explicit configuration
- **Ingestion vs. query time** — Process at ingestion for permanent actions (drop, mask, route, extract); process at query time for flexible analysis

---

Next Steps

Continue to **OPIPE-02: Span Processing & Enrichment** to configure OpenPipeline for distributed traces — filtering noisy spans, enriching attributes, and routing traces to dedicated buckets.

References

- OpenPipeline Documentation
- OpenPipeline Processing
- OpenPipeline Spans
- OpenPipeline Metrics
- Grail Bucket Management
- Data Security Context
